Natural Image Pretraining Improves Abstract Reasoning

Xiaoman Delores Ding, Keya Hu, Katelyn Gan, Victor Yin, and Kaiming He

Massachusetts Institute of Technology, Cambridge, MA 02139, USA

Abstract. Can visual representations learned from natural images transfer to abstract visual reasoning? We study this question on the Abstraction and Reasoning Corpus (ARC), where each task provides only a few input–output demonstrations and requires inferring a latent transformation rule. While many high-performing ARC systems rely on heavily pretrained language-model pipelines, we investigate whether visual self-supervised pretraining can benefit a vision-centric solver. We introduce Nat-ARC, a VARC-based pipeline that initializes its visual encoder from ImageNet-pretrained Masked Autoencoders before offline ARC training and per-task test-time adaptation. Across base, large, and huge model scales, ImageNet MAE pretraining consistently improves ARC-1 performance over matched randomly initialized baselines, with the best ImageNet-pretrained single model reaching $63.4 \pm 0.7\%$ pass@2. We further show that prediction-level ensembling across diverse training regimes, including no pretraining, ImageNet MAE pretraining, and ARC-style grid pretraining, reaches $70.2 \pm 0.6\%$ pass@2 on ARC-1 using a purely vision-based method. Together, these results suggest that visual self-supervised pretraining provides transferable representations for few-shot abstract grid reasoning.

1 Introduction

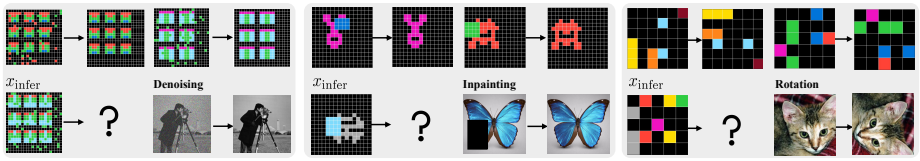


Fig. 1: Abstraction and Reasoning Corpus (ARC) with few-shot learning tasks. Each task typically has 2–5 input–output examples. ARC rules often involve real-world visual concepts, such as denoising [15], image inpainting [6], and rotation [20].

Unsupervised learning has become a cornerstone of modern computer vision, largely because it enables scalable representation learning from abundant

unlabeled data [8,9,22,23,33]. By pretraining on large collections of natural images, models can acquire rich perceptual primitives—such as objectness, spatial structure, and invariances—that transfer effectively to a wide range of downstream tasks. Beyond improving generalization, this form of pretraining can also mitigate overfitting, making it particularly valuable in regimes where labeled supervision is scarce or expensive.

In this work, we ask whether the benefits of representation learning from natural images extend to visual abstract reasoning, where supervision is limited—and often intrinsically insufficient—for learning task-specific rules from scratch. The Abstraction and Reasoning Corpus (ARC) exemplifies this challenge [10, 12]: each task provides only 2–5 demonstration input–output pairs, rendered as highly abstract 2D grids (up to 30×30 pixels with at most 10 colors), yet requires inferring a latent transformation rule and applying it to novel inputs, as shown in Fig. 1. This combination of extreme data scarcity and high combinatorial diversity makes ARC difficult: success typically requires systematic reasoning over object structure, geometry, symmetry, and compositional rules rather than pattern matching or memorization. ARC therefore serves as a strong testbed for assessing whether a model backbone can support few-shot *abstract reasoning* under minimal task-level supervision.

Because ARC combines highly compositional reasoning with only a few examples per task, nearly all mainstream high-performing approaches move beyond vision-centric learning and adopt LLM-centered pipelines [11,13]. These methods typically convert grids into symbolic [4,30,40] or textual representations [3,5,18] and then use an LLM, often coupled with iterative chain-of-thought rule induction or program synthesis, to produce an explicit transformation hypothesis and apply it to the inference input grid for verification.

At the same time, ARC tasks are presented as visual grids, and humans can often infer their rules directly from objects, spatial relations, and visual regularities. Motivated by this perspective, recent work has begun to revisit ARC through a vision-first paradigm that operates on grids end-to-end and emphasizes neural perception with iterative refinement. Early attempts such as ViT-ARC [31] explored this direction, while VARC [27] explicitly formalizes the vision-centric framing: treating ARC as conditional image-to-image translation, and achieving performance comparable to LLM-driven methods [27]. Building on this pipeline, LoopViT [35] incorporates recurrent training and inference within the same vision-centric framing, yielding substantially higher performance. More broadly, large-scale video-pretrained models have also exhibited strong few-shot generalization [1,2], further underscoring the potential of visual pretraining as a foundation for abstract reasoning on ARC. These results show that vision-centric ARC solving is viable.

A key remaining gap is a pretraining asymmetry between dominant ARC solvers. Language-centric pipelines typically start from massively pretrained models whose scaling behavior and transferability are central to their success [7, 14], whereas many vision-centric solvers still learn most of their representations from ARC supervision. This motivates our central question, directly aligned

with the promise of unsupervised visual pretraining: can real-world visual knowledge—acquired through large-scale unsupervised learning—transfer to abstract grid reasoning in ARC? Concretely, we ask whether incorporating vision pretraining into the VARC pipeline can close the pretraining gap and unlock more reliable scaling and transfer for vision-centric ARC solving.

To address this gap, we introduce *Natural-image ARC* (NAT-ARC), which adds an explicit unsupervised pretraining stage on natural images to the ARC pipeline. Specifically, we adopt Masked Autoencoders (MAE) [22] as our pretraining objective, pretraining vision backbones on large-scale unlabeled natural images (e.g. ImageNet [16] data) and then transferring the resulting encoder backbone to downstream ARC solving through relatively small-scale in-domain finetuning. Our NAT-ARC pipeline allows us to test, in a controlled way, whether natural image pretraining can improve generalization under scarce supervision, reduce overfitting to the limited ARC training set, and yield more stable few-shot adaptation at test time. Finally, we systematically scale model capacity to examine whether vision-centric ARC solvers exhibit LLM-like trends: whether larger pretrained models translate into stronger transfer and more reliable abstract reasoning on ARC.

Across these settings, we find that natural-image pretraining consistently benefits vision-centric ARC solving. ImageNet MAE pretraining improves ARC-1 performance over matched randomly initialized baselines across model scales, with the best single model reaching $63.4 \pm 0.7\%$ pass@2; these gains persist under repeated offline training and test-time adaptation. Prediction-level ensembling across no pretraining, ImageNet MAE pretraining, and ARC-style grid pretraining further reaches $70.2 \pm 0.6\%$ pass@2 on ARC-1. To better understand where pretraining helps, we analyze ImageNet-improved tasks and find recurring patterns involving visual matching, copying, and connected-component reasoning. While qualitative rather than a complete taxonomy, these results identify large-scale visual self-supervised pretraining as a simple and effective ingredient for vision-centric ARC solvers.

2 Related Work

Classic Approaches to ARC. The Abstraction and Reasoning Corpus (ARC) [10,12] is a widely used benchmark for evaluating fluid intelligence: the ability to acquire novel concepts and knowledge on the fly from only a few demonstrations. Given the scarcity of task-level supervision and the need for flexible rule induction, large language models (LLMs) have long been a natural choice for tackling ARC, owing to their strong few-shot learning capabilities [7]. A substantial body of work leverages LLMs in this setting, including fine-tuning pretrained LLMs on large-scale synthesized ARC-style data [3, 18, 30], using chain-of-thought reasoning to propose natural-language hypotheses [5, 41], and performing program synthesis to generate Python programs that capture the underlying rules [4, 34].

In parallel, another line of work explores training smaller models largely from scratch. One direction uses recurrent architectures [28, 39], aiming to capture

iterative processing and hierarchical updating. Others show that, by minimizing description length, compact models can be trained from scratch while achieving non-trivial performance [32]. Additional efforts include approaches such as self-supervised compression [38] and vision-centric formulations [27,35]. Collectively, these methods can reach competitive performance with substantially smaller models and lower compute budgets, offering an appealing alternative to LLM-heavy pipelines which are built on large language corpus pretraining.

Vision-centric ARC Solvers. Early efforts such as ViT-ARC [31] explored vision-based ARC solvers, and VARC [27] crystallized this direction by formulating ARC as conditional image-to-image translation. In VARC, each task provides supervised input–output demonstrations, and the model is conditioned (e.g., via a task token) to capture the underlying transformation; the approach follows a two-stage pipeline of offline training across tasks followed by per-task test-time training for adaptation. Building on this vision-centric framing, LoopViT [35] further incorporates recurrent training and inference and achieves substantially higher performance. More broadly, results from large-scale video-pretrained models also show strong few-shot generalization [1, 2], underscoring the promise of visual pretraining as a foundation for abstract reasoning. Our work directly builds on the VARC framework and asks whether adding an explicit pretraining stage can further improve performance and generalization.

Vision Pretraining. Pretraining refers to learning representations on large-scale data before training on a target task, with the goal of improving robustness and downstream performance. In vision, widely adopted pretraining paradigms include masked image modeling [22,42,43], contrastive learning [9,23], and distillation-based self-supervision [8, 33, 36].

Several recent works also highlight the role of self-supervised learning and representation learning for ARC. Vakde [38] trains a small Transformer with next-token prediction over stacked inputs and outputs, CompressARC [32] learns task-specific compression via a VAE-like objective, and Acuaviva *et al.* [2] show that fine-tuning a pretrained video diffusion model can outperform fine-tuning an LLM on ARC-style tasks. Motivated by these findings, we incorporate an explicit pretraining stage into the VARC pipeline, preceding offline training, to improve performance.

3 Background

3.1 ARC Problem Definition

The ARC benchmark consists of extremely few-shot visual reasoning tasks. Each task T defines a hidden mapping from an input grid x to an output grid y, where each grid has size up to 30×30 and uses at most 10 discrete colors. A task provides a small set of demonstration pairs $\mathcal{D}^T_{\text{demo}} = \{(x_i, y_i)\}_{i=1}^m$, typically with 2–4 examples, and one or more inference inputs x_{infer}. At test time, the

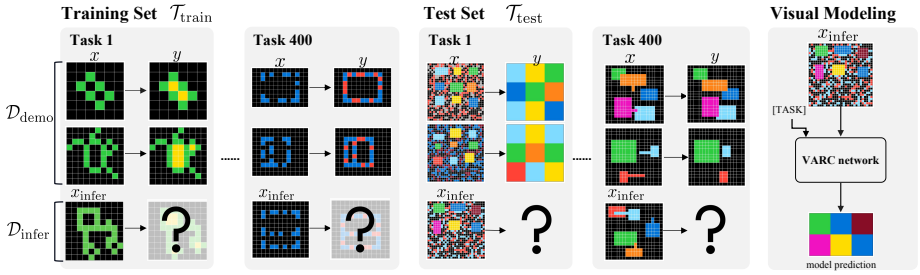


Fig. 2: The ARC problem definition. We follow the formulation from VARC [27]. ARC contains hundreds of extremely few-shot grid reasoning tasks. Each task T specifies a hidden mapping from input grid x to output grid y, and includes demonstrations $\mathcal{D}^T_{\text{demo}}$ (2–4 pairs) plus inference examples $\mathcal{D}^T_{\text{infer}}$ (1–2 pairs). At test time, the model uses $\mathcal{D}^T_{\text{demo}}$ and x_{infer} to predict y_{infer}. Tasks are split into $\mathcal{T}_{\text{train}}$ (all pairs available for offline training) and unseen $\mathcal{T}_{\text{eval}}$ (demonstration pairs and inference inputs only).

model must infer the underlying transformation from $\mathcal{D}^T_{\text{demo}}$ and predict the corresponding output y_{infer}.

ARC tasks are split into training tasks $\mathcal{T}_{\text{train}}$ and held-out evaluation tasks $\mathcal{T}_{\text{eval}}$. For $\mathcal{T}_{\text{train}}$, all input–output pairs are available for offline training. For $\mathcal{T}_{\text{eval}}$, only the demonstration pairs and inference inputs are available during adaptation, and predictions are evaluated by exact grid match.

3.2 VARC Formulation

Our work builds on VARC [27], which formulates ARC as conditional image-to-image translation. Each ARC grid is placed on a fixed-size canvas, allowing the model to operate on grid tasks with a standard vision backbone. A learnable task embedding identifies the current task, and the model predicts an output grid conditioned on both the input grid and this task embedding:

$$\mathcal{L}(\theta) = -\mathbb{E}_{T,i}\big[\mathcal{D}(y_i, f_\theta(x_i \mid T))\big]. \tag{1}$$

A prediction is counted as correct only if it exactly matches the ground-truth output grid.

VARC trains in two stages. First, a model is trained offline across the ARC training tasks. Second, at evaluation time, the model adapts to each unseen task using its few demonstration examples through test-time training. VARC also uses simple grid-preserving augmentations, such as translation, scaling, rotation, flipping, and color permutation, to improve generalization. NAT-ARC keeps this vision-centric formulation but adds an explicit visual pretraining stage before offline ARC training.

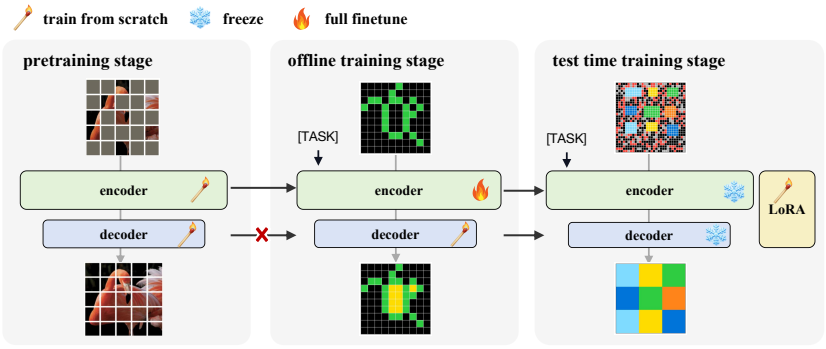


Fig. 3: Three training stages in our NAT-ARC pipeline. (*Left*) Pretraining: a masked autoencoder is trained on unlabeled natural images (e.g. ImageNet). (*Middle*) Offline training: the encoder is initialized from the pretraining stage and fine-tuned on ARC, while the decoder is trained from scratch. (*Right*) Test-time training: the backbone is frozen and task-specific LoRA parameters and task embeddings are optimized from scratch.

4 Method

4.1 NAT-ARC Pipeline and Visual Pretraining

NAT-ARC extends the VARC pipeline by adding a visual pretraining stage before offline ARC training. The resulting pipeline has three stages, as shown in Fig. 3: (i) pretraining a visual encoder on unlabeled images, (ii) offline training on ARC tasks with the pretrained encoder and a randomly initialized decoder, and (iii) per-task test-time adaptation on unseen ARC tasks.

We primarily use self-supervised Masked Autoencoder (MAE) pretraining [22] on ImageNet [16]. MAE trains an encoder–decoder Vision Transformer by masking a large fraction of image patches and reconstructing the missing pixels from the visible patches. This objective encourages the encoder to learn transferable visual representations without labels.

Although MAE is commonly applied to natural images, the objective is domain-agnostic. We therefore also evaluate MAE pretraining on unlabeled ARC-style grids. In the supplementary material, we further evaluate DINOv2-small [33] as an alternative ImageNet-pretrained encoder to test whether the transfer effect extends beyond masked-autoencoding pretraining.

4.2 NAT-ARC Architecture

NAT-ARC keeps the vision-centric image-to-image formulation of VARC, but uses an MAE-compatible encoder–decoder Transformer so that pretrained visual encoders can be transferred into the ARC pipeline. The model consists of input/output layers, an encoder, and a lightweight decoder.

Table 1: Encoder configurations and corresponding total model parameter counts.

size	enc depth	enc embed dim	enc num heads	#params
base	12	768	12	114M
large	24	1024	16	334M
huge	32	1280	16	664M

Input and Output Layers. The input and output layers interface between the discrete ARC canvas and the model's latent space. We place each ARC grid onto a 64×64 canvas and map each discrete color index to a learnable embedding. The canvas is partitioned into non-overlapping 2×2 patches, which are flattened into a sequence of tokens. We add positional embeddings to preserve 2D structure and prepend a learned task token derived from a task embedding. For prediction, the output head linearly projects decoder latents to logits over the discrete color vocabulary.

Encoder–Decoder Transformer. To align with MAE initialization, Nat-ARC adopts an encoder–decoder architecture in which both modules are stacks of bidirectional Transformer blocks. The encoder and decoder are connected by a normalization layer and a linear projection to handle dimension mismatch. Following VARC, we use 2D rotary positional embeddings (RoPE) [24] in all Transformer blocks to strengthen spatial awareness. We study scalability by varying only the encoder size, as shown in Tab. 1, while keeping the decoder lightweight and fixed across all experiments.

4.3 Training and Adaptation Details

ImageNet MAE Initialization. Large-scale pretraining on natural images is computationally expensive. Rather than training from scratch, we leverage publicly available checkpoints from the original MAE work [22], pretrained on ImageNet-1K [16]. Under this setup, the pretraining stage requires no additional optimization: we initialize the encoder from an MAE checkpoint and proceed directly to offline ARC training.

When transferring public ImageNet MAE checkpoints into Nat-ARC, we must resolve two architectural mismatches. First, public ImageNet MAE checkpoints use larger image sizes and patch sizes than our ARC canvas, so we discard the pretrained patch and absolute positional embeddings and transfer only the encoder backbone weights. Second, public MAE checkpoints do not use 2D RoPE, whereas VARC shows that 2D RoPE is important for ARC performance [27]. We verified with a base-scale ImageNet MAE run that enabling RoPE gives comparable offline ARC performance, so we use public checkpoints throughout. More details are provided in the supplementary material.

ARC-Style Grid MAE Pretraining. For in-domain ARC-style pretraining, we build an unlabeled corpus from three sources: the original 400 ARC-1 training tasks, the RE-ARC [25] dataset, and NVARC [37] with LLM-synthesized ARC-like tasks. RE-ARC contributes roughly 400k grid pairs, while NVARC provides 1.5M pairs. We pretrain with the MAE objective for 500 epochs on the combined corpus using a mask ratio of 75%. Because the tokenization matches the downstream ARC setup, we retain the learned patch embedding, positional embedding, and encoder from the pretraining stage.

Offline ARC Training. After initialization, we train on the ARC-1 training tasks together with 400k examples from RE-ARC [25]. For the original ARC-1 training tasks, only the demonstration/example pairs are used for offline training; the corresponding inference inputs and outputs are reserved for offline-stage evaluation.

For ImageNet MAE initialization, the encoder backbone is initialized from the public checkpoint as described above; for ARC-style grid pretraining, the pretrained ARC-grid encoder and embeddings are retained; for no-pretraining baselines, the same architecture is randomly initialized. In all cases, the decoder is trained from scratch. Following VARC, we apply data augmentations including random resolution changes and translations, and train for 200 epochs in all experiments.

Test-Time Adaptation. At evaluation time, we adapt the model separately to 400 unseen ARC-1 evaluation tasks. Following VARC, we construct auxiliary tasks via reversible transformations, including flips, rotations, and color permutations; each auxiliary task is assigned its own learnable task embedding during adaptation.

As we scale to substantially larger models than those in VARC, parameter-efficient adaptation becomes crucial. Our best-performing recipe uses low-rank adaptation (LoRA) [26]. Concretely, for each task we optimize (i) task embeddings initialized from scratch and (ii) LoRA adapters inserted into attention projections and MLP layers, while keeping the encoder and decoder trained in the offline stage frozen. Additional ablations over TTT hyperparameters are provided in the supplementary material.

4.4 Inference and Prediction-Level Ensembling

Task-agnostic augmentations allow us to generate multiple candidate predictions for the same test input x_{infer}. For each augmented "view" v (e.g., different translations, scales, and rotations on the canvas), we run the model to obtain a prediction $\hat{y}^{(v)} = f_\theta(x_{\text{infer}}^{(v)}; T)$ and map it back to the original grid by aggregating canvas pixels.

For each task T, we sample multiple random views and obtain a set of candidates $\{\hat{y}^{(v)}\}$. We then consolidate these candidates via majority voting, based on the assumption that the correct output y_{infer} exhibits the highest cross-view

agreement. Two predictions are considered *consistent* only if their output grids match *exactly*. We select the output that matches the largest number of other predictions, and report pass@2 by keeping the top-2 most supported modes.

Because voting operates at the prediction level, the same procedure naturally extends to ensembles. We pool candidate outputs from multiple independently trained models and apply the same majority-voting rule. This allows us to evaluate whether pretraining improves not only single-model performance but also the quality of candidate predictions available under prediction-level aggregation.

5 Experiments

5.1 Experiment Setup

Because ARC-1 contains only 400 evaluation tasks and each prediction is scored as either correct or incorrect, per-task accuracy is a binary outcome and a certain degree of variance across runs is unavoidable. To distinguish genuine gains from stochastic variation in our main ImageNet-pretraining comparisons, we report results over repeated, independent trials with different random seeds. For ImageNet MAE pretraining and no-pretraining baselines, we train 5 independent offline checkpoints for each model scale and training configuration. Each checkpoint is evaluated with 2 independent test-time training runs, and we average the two ARC-1 scores to obtain one checkpoint-level score. We report the mean and standard deviation over the resulting 5 checkpoint-level scores.

For ensemble results, we form ensembles directly at the prediction level by pooling candidate predictions from different offline checkpoints. Since prediction-level ensembling requires a fixed set of candidate outputs from each model, we use 2 independent TTT runs per checkpoint for these ensemble experiments and report the mean and standard deviation over the resulting ensemble combinations. ARC-style grid pretraining is included as an auxiliary comparison: due to its substantially higher pretraining cost, we use one offline checkpoint for each model scale, so the corresponding single-model points in Fig. 5 report performance without checkpoint-level error bars.

5.2 Offline Training Dynamics

We first study whether natural-image pretraining improves offline optimization for ARC. Concretely, we compare models trained from scratch against the same architectures initialized from an ImageNet-pretrained checkpoint. Fig. 4 shows the offline-stage evaluation curves from one representative checkpoint run at each model scale. During offline training, we observe a consistent advantage from ImageNet pretraining across base, large, and huge models: it yields markedly faster convergence with substantially higher accuracy in early epochs and improves final performance.

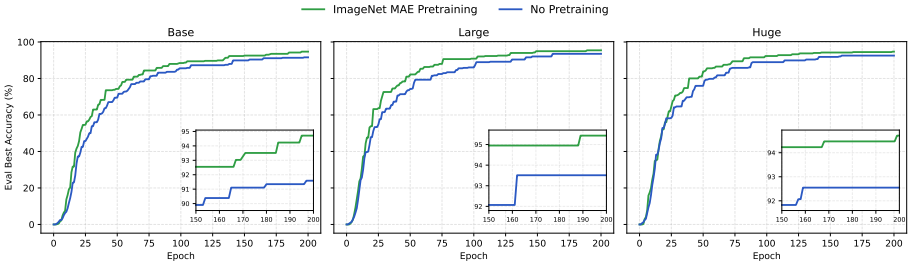


Fig. 4: Best offline-stage evaluation accuracy on the ARC-1 training dataset across model scales. We compare no-pretraining and ImageNet-pretrained models at base, large, and huge scales. Each curve shows one representative offline-training run for the corresponding model scale and pretraining condition. The last 50 epochs are zoomed in for better visibility. Pretrained initializations converge faster and typically reach higher accuracy than training from scratch in the offline stage.

5.3 Final ARC-1 Performance Across Scales

We next evaluate whether the offline-training advantages from Sec. 5.2 translate into final performance on unseen ARC-1 evaluation tasks after test-time training. For each model scale, we compare ImageNet MAE initialization against a matched no-pretraining baseline using the same architecture, offline training recipe, and test-time adaptation procedure.

As shown in Fig. 5 (left), ImageNet MAE pretraining consistently improves ARC-1 pass@2 across base, large, and huge encoders. These gains persist after per-task adaptation, indicating that the pretrained representations are not only useful for offline optimization but also improve final few-shot generalization to unseen ARC tasks.

The scaling trend further highlights the benefit of pretraining. Without pretraining, performance improves from base to large but slightly degrades at the huge scale, suggesting that simply increasing model capacity is not sufficient in this low-data regime. In contrast, ImageNet-pretrained models improve more consistently with encoder size. These results suggest that natural-image pretraining provides a useful inductive bias that makes scaling vision-centric ARC solvers more reliable.

5.4 Prediction-Level Ensembles

Beyond single-model accuracy, we evaluate whether the benefits of pretraining persist under prediction-level ensembling. Our inference procedure already produces multiple candidate outputs for each model through task-agnostic augmentations; to ensemble multiple models, we pool their candidate predictions and apply the same majority-voting rule described in Sec. 4.4.

The middle panel of Fig. 5 compares ensembles formed from independently trained offline checkpoints. Ensembling two non-pretrained checkpoints improves

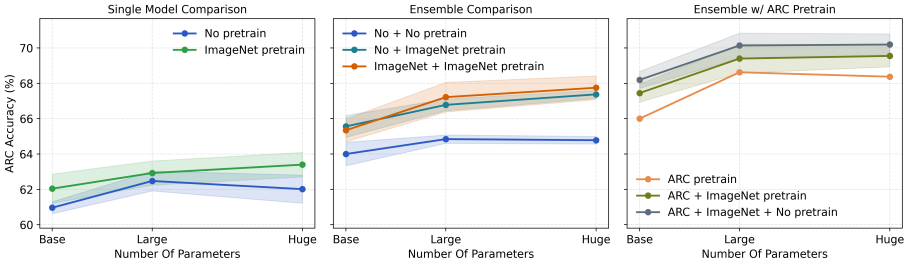


Fig. 5: ARC-1 performance and prediction-level ensembling across pretraining regimes. (*Left*) ImageNet MAE pretraining consistently improves single-model ARC-1 pass@2 over matched randomly initialized baselines across model scales. (*Middle*) Prediction-level ensembles formed from independently trained checkpoints improve performance. Ensembles that include ImageNet-pretrained models outperform ensembles formed only from non-pretrained models, while two-ImageNet and mixed scratch–ImageNet ensembles perform similarly. (*Right*) ARC-style grid pretraining further improves performance, and combining predictions from no pretraining, ImageNet MAE pretraining, and ARC-style grid pretraining gives the best ensemble result.

over a single non-pretrained model, showing that checkpoint diversity is useful. Ensembles containing ImageNet-pretrained checkpoints perform better than scratch-only ensembles, while two-ImageNet and mixed scratch–ImageNet ensembles perform similarly. This suggests that the ensemble gain largely comes from the stronger candidate predictions produced by ImageNet-pretrained models, and that the benefit of natural-image pretraining persists under prediction-level aggregation.

Finally, we evaluate pretraining on in-domain, unlabeled ARC-style grids (Fig. 5, right). ARC-style pretraining further improves downstream performance, although the single-checkpoint results do not show further improvement from large to huge scale. The strongest ensemble is obtained by pooling predictions from all three training regimes: no pretraining, ImageNet MAE pretraining, and ARC-style grid pretraining. This result shows that prediction-level aggregation can benefit from diverse visual solvers, but we report single-model ImageNet-pretrained performance separately as the main measure of natural-image transfer.

5.5 System-Level Comparison

We contextualize NAT-ARC relative to representative prior systems in Tab. 2. These systems differ substantially in model type, training data, model access, inference-time computation, and use of test-time adaptation, so the table should be read as a contextual comparison rather than a direct ranking. NAT-ARC improves a vision-centric VARC-style pipeline through natural-image and ARC-style grid pretraining, while a clear gap remains to the strongest specialized and proprietary systems.

Table 2: System-level comparison on ARC-1 and ARC-2. Systems are grouped by broad method family and source of large-scale external pretraining. The table reports mean scores only; NAT-ARC mean±std values are 63.4±0.7 / 8.6±0.8 for the single model and 70.2±0.6 / 13.1±0.7 for the ensemble on ARC-1 / ARC-2.

System	# Params	ARC-1	ARC-2
Human reference			
Avg. human [29]	–	60.2	–
Best human [17]	–	98.0	100.0
General-purpose LLMs (language pretrain)			
DeepSeek-R1 [21]	671B	15.8	1.3
GPT-5 [17]	–	44.0	1.9
Gemini 3 Pro [17]	–	98.0	77.1
ARC-specialized LLM systems (language pretrain)			
TTT+BARC [3, 30]	16B	62.8	–
The ARChitects [18]	8B	71.6	–
Neural systems (no external pretrain)			
HRM [39]	27M	40.3	5.0
TRM [28]	7M	44.6	7.8
Vision-centric systems (no external pretrain)			
VARC [27]	19M	54.0	8.3
LoopViT [35]	18M	65.8	14.2
Loop-OWM [19]	10.6M	68.5	22.5
NAT-ARC (ImageNet / ARC-style MAE pretrain)			
NAT-ARC	0.6B	63.4	8.6
NAT-ARC ensemble	2B	70.2	13.1

6 Analysis

6.1 Attention Patterns

To move beyond aggregate metrics and better understand model behavior, we visualize attention patterns in Fig. 6. For a given ARC task, we compute the total attention received by each patch, averaged across all attention heads and aggregated over encoder and decoder layers.

In the top row, we show attention maps for encoder layers 8 and 12 at offline training epoch 0 (i.e., initialization) for three base models with different pretraining strategies. The no-pretraining model exhibits nearly uniform attention, whereas both MAE-pretrained models already show an emergent tendency to distinguish foreground structure from background. This effect is particularly striking for ImageNet pretraining, since the model has not been exposed to ARC tasks prior to offline training.

In the bottom row, we show attention maps for encoder layer 12 and decoder layer 8 (the final decoder layer before the output head) at epoch 50. At this

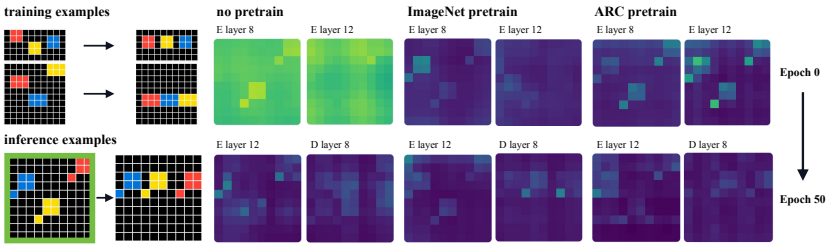


Fig. 6: Attention patterns under three pretraining strategies (no pretraining, ImageNet MAE pretraining, and ARC-style grid MAE pretraining) at offline training epochs 0 (*top*) and 50 (*bottom*). "E" denotes encoder layers and "D" denotes decoder layers. Both ImageNet and ARC-style grid MAE pretraining exhibit more structured attention, focusing on patches that are more relevant for reasoning.

point, the decoder attention of the no-pretraining model remains diffuse, while the pretrained models concentrate more clearly on foreground regions in the output. This aligns with our findings in Sec. 5.2: pretraining provides a useful prior and accelerates learning during offline training.

6.2 Task-level Effect of ImageNet Pretraining

The performance gap between two-ImageNet and two-scratch ensembles in Fig. 5 (middle) motivates a task-level analysis of where ImageNet pretraining helps. Using the huge-model runs from Sec. 5.1, we form all pairwise two-checkpoint ensembles within the 5 ImageNet MAE-pretrained checkpoints and within the 5 matched no-pretraining checkpoints, yielding 10 two-ImageNet and 10 two-scratch ensembles. We then compute, for each ARC-1 evaluation task, the fraction of ensembles in each family that solve the task.

To avoid selecting examples manually, we first identify a large-gain subset using only solve rates. Specifically, we select tasks solved by at most 30% of two-scratch ensembles and by at least 70% of two-ImageNet ensembles. This criterion selects 15 tasks. We then manually annotate all selected tasks according to their dominant transformation pattern. Among them, 6 involve match-and-copy behavior, where the solver must identify a matching object, color, or pattern and transfer the corresponding structure to the output. Another 6 involve connected-component reasoning, such as identifying, filling, or recoloring spatially connected regions. The remaining 3 tasks do not fit cleanly into these two recurring categories. The full set of 15 tasks, together with solve rates and manual labels, is provided in the supplementary material.

Fig. 7 shows representative examples from these two recurring categories. These examples suggest that ImageNet pretraining may provide useful visual biases for grouping, matching, and region-level segmentation, even in abstract grid domains. We emphasize that this analysis is qualitative, and the manual labels are not intended to provide an exhaustive taxonomy of ARC tasks. Rather,

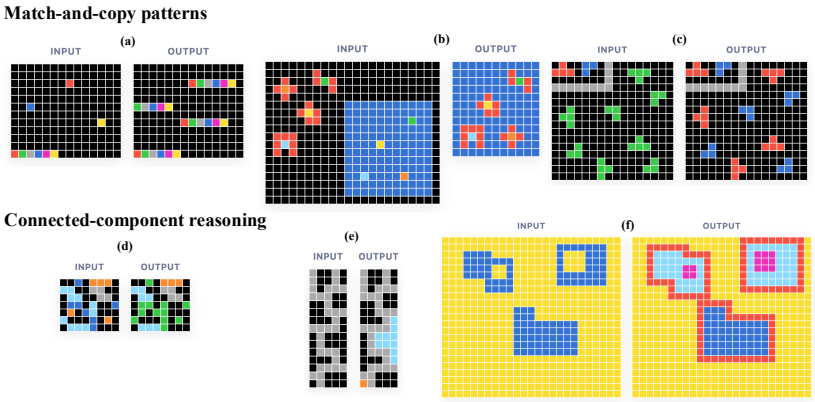


Fig. 7: Representative ARC tasks improved by ImageNet pretraining. Each panel shows one input–output demonstration pair from a task solved reliably by ImageNet-pretrained models but not by models trained from scratch. The examples are grouped into two recurring categories: **match-and-copy** tasks (top), which require identifying a matching object, color, or pattern and transferring the associated structure to the output; and **connected-component reasoning** tasks (bottom), which require detecting, filling, or recoloring spatially connected regions. Detailed task IDs and rule descriptions are provided in the supplementary material.

they offer an interpretable view of the task types most improved by ImageNet pretraining under our selection criterion.

6.3 ARC Transformations on Natural Images

To probe the relationship between natural images and ARC puzzles, we conduct a toy experiment that applies ARC-style transformations to ImageNet images. The pipeline is illustrated in Fig. 8.

We first train an autoencoder that maps an ImageNet image to a discrete 60×60 latent grid with vocabulary size 10, which we treat as an ARC-like representation. Both the encoder and decoder are Transformer-based. The decoder reconstructs the original image from the latent grid, yielding an encoder that effectively projects natural images into the ARC domain. We then place the 60×60 latent grid at a random location on a 64×64 canvas and feed it into our NAT-ARC model, conditioned on a selected task token. Finally, we interpret the predicted ARC output as a latent grid and decode it back into pixel space. As shown in Fig. 8, this end-to-end pipeline enables ARC transformations to be applied to natural images.

We use a cat image from ImageNet-1K as a representative example and consider two ARC-style tasks: rotating the input by 180° and adding a blue border. Despite mild blur and discretization noise introduced by the autoencoder, the model produces the expected rotated cat and correctly adds the border. Notably, we do not explicitly align the autoencoder latent space with the original

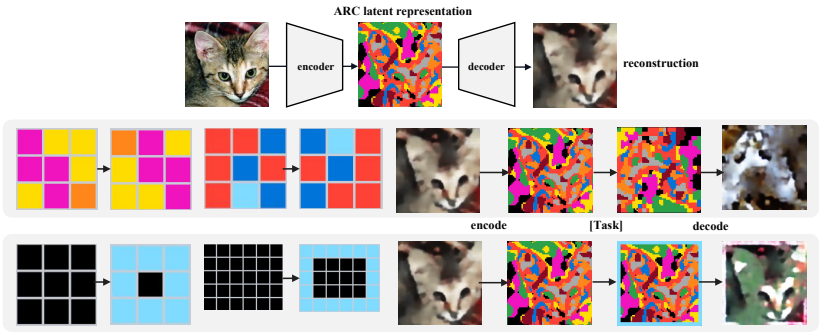


Fig. 8: Applying ARC transformations to natural images. (*Top*) Autoencoder architecture used to learn a discrete ARC-like latent representation. (*Middle*) An ARC task rotates the input by 180°, and the same rotation is applied to a cat image. (*Bottom*) An ARC task adds a border to the input, and the same border is added to the natural image.

ARC grid space, making this transfer non-trivial. These examples illustrate that Nat-ARC can apply learned ARC-style transformations to a natural-image latent representation, despite not being explicitly trained for this setting. More broadly, this toy experiment suggests a possible connection between abstract grid transformations and transformations over natural-image representations.

7 Conclusions

We empirically study whether visual self-supervised pretraining on natural images can transfer to abstract grid reasoning in ARC. By adding an ImageNet MAE pretraining stage to the VARC pipeline, we find that pretrained visual encoders consistently improve final ARC-1 performance over matched randomly initialized baselines across model scales. These gains persist after offline ARC training and per-task test-time adaptation, suggesting that natural-image pretraining provides useful visual representations and optimization priors for few-shot ARC solving. Prediction-level ensembling further improves performance when combining models trained under diverse regimes, including no pretraining, ImageNet MAE pretraining, and ARC-style grid pretraining. Our task-level and attention analyses suggest that pretraining benefits tasks involving visual matching, copying, and region-level structure, though they do not provide a complete mechanistic explanation. Overall, our results identify large-scale visual self-supervised pretraining as a simple and effective ingredient for improving vision-centric ARC solvers.

References

1. Acuaviva, P., Davtyan, A., Hassan, M., Stapf, S., Rahimi, A., Alahi, A., Favaro, P.: From generation to generalization: Emergent few-shot learning in video diffusion models. arXiv:2506.07280 (2025), https://arxiv.org/abs/2506.07280
2. Acuaviva, P., Davtyan, A., Hassan, M., Stapf, S., Rahimi, A., Alahi, A., Favaro, P.: Rethinking visual intelligence: Insights from video pretraining. CoRR **abs/2510.24448** (2025)
3. Akyürek, E., Damani, M., Zweiger, A., Qiu, L., Guo, H., Pari, J., Kim, Y., Andreas, J.: The surprising effectiveness of test-time training for few-shot learning. In: ICML (2025), https://openreview.net/forum?id=asgBo3FNdg
4. Berman, J.: How I got a record 53.6% on ARC-AGI. Substack (2024), https://jeremyberman.substack.com/p/how-i-got-a-record-536-on-arc-agi, accessed: 2025-10-13
5. Berman, J.: How I got the highest score on ARC-AGI again swapping Python for English. Substack (2025), https://jeremyberman.substack.com/p/how-i-got-the-highest-score-on-arc-agi-again
6. Bertalmio, M., Sapiro, G., Caselles, V., Ballester, C.: Image inpainting. In: Proceedings of the 27th annual conference on Computer graphics and interactive techniques. pp. 417–424 (2000)
7. Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al.: Language models are few-shot learners. Advances in neural information processing systems **33**, 1877–1901 (2020)
8. Caron, M., Touvron, H., Misra, I., Jégou, H., Mairal, J., Bojanowski, P., Joulin, A.: Emerging properties in self-supervised vision transformers. In: 2021 IEEE/CVF International Conference on Computer Vision, ICCV 2021, Montreal, QC, Canada, October 10-17, 2021. pp. 9630–9640. IEEE (2021)
9. Chen, T., Kornblith, S., Norouzi, M., Hinton, G.E.: A simple framework for contrastive learning of visual representations. In: ICML. Proceedings of Machine Learning Research, vol. 119, pp. 1597–1607. PMLR (2020)
10. Chollet, F.: On the measure of intelligence. arXiv:1911.01547 (2019)
11. Chollet, F., Knoop, M., Kamradt, G., Landers, B.: ARC Prize 2024: Technical report. arXiv:2412.04604 (2024), https://arxiv.org/abs/2412.04604
12. Chollet, F., Knoop, M., Kamradt, G., Landers, B., Pinkard, H.: ARC-AGI-2: A new challenge for frontier AI reasoning systems. arXiv:2505.11831 (2025), https://arxiv.org/abs/2505.11831
13. Chollet, F., Knoop, M., Kamradt, G., Landers, B.: Arc prize 2025: Technical report. arXiv:2601.10904 (2026)
14. Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H.W., Sutton, C., Gehrmann, S., et al.: Palm: Scaling language modeling with pathways. Journal of Machine Learning Research **24**(240), 1–113 (2023)
15. Dabov, K., Foi, A., Katkovnik, V., Egiazarian, K.: Image denoising by sparse 3-d transform-domain collaborative filtering. IEEE Transactions on image processing **16**(8), 2080–2095 (2007)
16. Deng, J., Dong, W., Socher, R., Li, L.J., Li, K., Fei-Fei, L.: Imagenet: A large-scale hierarchical image database. In: 2009 IEEE conference on computer vision and pattern recognition. pp. 248–255. Ieee (2009)
17. Foundation, A.P.: ARC-AGI benchmarking: Leaderboard and dataset for the ARC-AGI benchmark. https://arcprize.org/leaderboard (2025), accessed: 2025-11-01

18. Franzen, D., Disselhoff, J., Hartmann, D.: Product of experts with LLMs: Boosting performance on ARC is a matter of perspective. arXiv:2505.07859 (2025), https://arxiv.org/abs/2505.07859
19. Gao, G., Schölkopf, B., Geiger, A.: Slots, transitions, loops: Learning composable world models for arc (2026), https://arxiv.org/abs/2606.12316
20. Gidaris, S., Singh, P., Komodakis, N.: Unsupervised representation learning by predicting image rotations. arXiv preprint arXiv:1803.07728 (2018)
21. Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al.: Deepseek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. arXiv:2501.12948 (2025), https://arxiv.org/abs/2501.12948
22. He, K., Chen, X., Xie, S., Li, Y., Dollár, P., Girshick, R.B.: Masked autoencoders are scalable vision learners. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022. pp. 15979–15988. IEEE (2022). https://doi.org/10.1109/CVPR52688.2022.01553, https://doi.org/10.1109/CVPR52688.2022.01553
23. He, K., Fan, H., Wu, Y., Xie, S., Girshick, R.B.: Momentum contrast for unsupervised visual representation learning. In: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2020, Seattle, WA, USA, June 13-19, 2020. pp. 9726–9735. Computer Vision Foundation / IEEE (2020)
24. Heo, B., Park, S., Han, D., Yun, S.: Rotary position embedding for vision transformer. In: ECCV (10). Lecture Notes in Computer Science, vol. 15068, pp. 289–305. Springer (2024)
25. Hodel, M.: Addressing the abstraction and reasoning corpus via procedural example generation. CoRR **abs/2404.07353** (2024)
26. Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W.: Lora: Low-rank adaptation of large language models. In: ICLR. OpenReview.net (2022)
27. Hu, K., Cy, A., Qiu, L., Ding, X.D., Wang, R., Zhu, Y.E., Andreas, J., He, K.: ARC is a vision problem! CoRR **abs/2511.14761** (2025)
28. Jolicoeur-Martineau, A.: Less is more: Recursive reasoning with tiny networks. arXiv:2510.04871 (2025), https://arxiv.org/abs/2510.04871
29. LeGris, S., Vong, W.K., Lake, B.M., Gureckis, T.M.: H-ARC: A robust estimate of human performance on the abstraction and reasoning corpus benchmark. arXiv:2409.01374 (2024), https://arxiv.org/abs/2409.01374
30. Li, W., Hu, K., Larsen, C., Wu, Y., Alford, S., Woo, C., Dunn, S.M., Tang, H., Zheng, W., Pu, Y., Ellis, K.: Combining induction and transduction for abstract reasoning. In: ICLR. OpenReview.net (2025)
31. Li, W., Xu, Y., Sanner, S., Khalil, E.B.: Tackling the abstraction and reasoning corpus with vision transformers: the importance of 2D representation, positions, and objects. arXiv:2410.06405 (2024), https://arxiv.org/abs/2410.06405
32. Liao, I., Gu, A.: ARC-AGI without pretraining. https://iliao2345.github.io/blog_posts/arc_agi_without_pretraining/arc_agi_without_pretraining.html (2025)
33. Oquab, M., Darcet, T., Moutakanni, T., Vo, H.V., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A., Assran, M., Ballas, N., Galuba, W., Howes, R., Huang, P., Li, S., Misra, I., Rabbat, M., Sharma, V., Synnaeve, G., Xu, H., Jégou, H., Mairal, J., Labatut, P., Joulin, A., Bojanowski, P.: Dinov2: Learning robust visual features without supervision. Trans. Mach. Learn. Res. **2024** (2024)

34. Pang, E.: [arc-agi-2 sota] efficient evolutionary program synthesis. Substack (2025), https://ctpang.substack.com/p/arc-agi-2-sota-efficient-evolutionary
35. Shu, W.J., Qiu, X., Zhu, R.J., Chen, H.H., Liu, Y., Yang, H.: Loopvit: Scaling visual arc with looped transformers. arXiv preprint arXiv:2602.02156 (2026)
36. Siméoni, O., Vo, H.V., Seitzer, M., Baldassarre, F., Oquab, M., Jose, C., Khalidov, V., Szafraniec, M., Yi, S.E., Ramamonjisoa, M., Massa, F., Haziza, D., Wehrstedt, L., Wang, J., Darcet, T., Moutakanni, T., Sentana, L., Roberts, C., Vedaldi, A., Tolan, J., Brandt, J., Couprie, C., Mairal, J., Jégou, H., Labatut, P., Bojanowski, P.: Dinov3. CoRR **abs/2508.10104** (2025)
37. Sorokin, I., Puget, J.F.: Nvarc solution to arc-agi-2 2025. https://github.com/1ytic/NVARC/blob/main/nvarc_2025.pdf (2025)
38. Vakde, M.: New pareto frontier on arc-agi. Github (2025), https://mvakde.github.io/blog/new-pareto-frontier-arc-agi/, accessed: 2025-12-18
39. Wang, G., Li, J., Sun, Y., Chen, X., Liu, C., Wu, Y., Lu, M., Song, S., Yadkori, Y.A.: Hierarchical reasoning model. arXiv:2506.21734 (2025), https://arxiv.org/abs/2506.21734
40. Wang, R., Zelikman, E., Poesia, G., Pu, Y., Haber, N., Goodman, N.D.: Hypothesis search: Inductive reasoning with language models. In: ICLR (2024), https://openreview.net/forum?id=G7UtIGQmjm
41. Xu, Y., Li, W., Vaezipoor, P., Sanner, S., Khalil, E.B.: Llms and the abstraction and reasoning corpus: Successes, failures, and the importance of object-based representations (2023), https://arxiv.org/abs/2305.18354
42. Yang, L., Li, S., Li, Y., Lei, X., Wang, D., Mohamed, A., Zhao, H., Xu, H.: In pursuit of pixel supervision for visual pre-training. CoRR **abs/2512.15715** (2025)
43. Zhou, J., Wei, C., Wang, H., Shen, W., Xie, C., Yuille, A.L., Kong, T.: ibot: Image BERT pre-training with online tokenizer. CoRR **abs/2111.07832** (2021)